



**Faculty of Engineering and Technology
Computer Science Department**

COMP338 - Artificial Intelligence

Course Project 1

Prepared by:

Wasfia Awwad (1211039)

Supervised by:

Dr.Mohammed Hilal

First Semester

December 22, 2025

Table Of Contents

1. Introduction	2
2. Background on Genetic Algorithms	3
2.1 Steps to create a genetic algorithm	3
2.2 Parameters	3
3. Problem Formulation	3
3.1 Solution Representation	3
3.2 Chromosome Representation	3
3.3 Gene Representation	4
3.4 Search Space	4
3.5 Fitness Function Definition	4
4. Implementation Details	5
4.1. Core Components	5
4.2. Evolutionary Stages in Code	5
4.3. Multi-Threading & UI Synchronization	6
4.4. Termination Conditions	6
5. Parameter Tuning Analysis	6
5.1 Experiment 1:	6
5.2 Experiment 2	7
5.3 Experiment 3	8
5.4 Experiment 4	9
5.5 Experiment 5	10
5.6 Experiment 6	11
6. Results and Discussion	12
6.1 Analysis of Mutation Rate Impact (Exp 1, 2, 3, and 6)	12
6.2 Analysis of Population Size Impact (Exp 2, 4, and 5)	12
6.3 Key Findings and Observations	13
6.4 Conclusion	13
7. Screenshots	13
7.1 Main Application Window (Welcome Window)	13
7.2 Simulation (Main App)	14
7.3 Report (Statistics Window)	15
7.4 run() Method	16
7.5 tournamentSelection()	16
7.6 crossover(Chromosome partner)	17
7.7 mutate(double mutationRate)	17
7.8 calculateFitness(int[] passcode)	18
8. References	18

Table Of Figures

Figure 5.1.1 : Fitness Convergence for experiment 1	8
Figure 5.1.2 : Statistics for experiment 1	8
Figure 5.2.1 : Fitness Convergence for experiment 2	9
Figure 5.2.2 : Statistics for experiment 2	9
Figure 5.3.1 : Fitness Convergence for experiment 3	10
Figure 5.3.2 : Statistics for experiment 3	10
Figure 5.4.1 : Fitness Convergence for experiment 4	11
Figure 5.4.2 : Statistics for experiment 4	11
Figure 5.5.1 : Fitness Convergence for experiment 5	12
Figure 5.5.2 : Statistics for experiment 5	12
Figure 5.5.3 : Fitness Convergence for experiment 6	13
Figure 7.1 : Main Window	15
Figure 7.2 : Simulation Window	16
Figure 7.3 : Statistics Window	17
Figure 7.4 : Running Method	17
Figure 7.5 : tournamentSelection Selection Method	18
Figure 7.6 : crossover method	18
Figure 7.7 : Mutation Method	19
Figure 7.8 : Fitness Calculation Method	19

1. Introduction

Genetic Algorithms are search and optimization techniques inspired by the principles of natural evolution, such as selection, crossover, and mutation. They are powerful in solving problems with large and complex search spaces where random search would consume an insane amount of time. By iteratively evolving a population of solutions, Genetic Algorithms are able to get an optimal or near-optimal solution.

This project is about implementing a Genetic Algorithm to simulate an attempt at breaking a password represented as a randomly generated 32-bit binary sequence. The program begins by generating a hidden passcode consisting of 32 binary digits. An initial population of randomly generated candidate solutions is then created, and the Genetic Algorithm is applied to evolve these candidates over successive generations until the correct passcode is discovered.

The objective of this project is to design and implement all stages of the Genetic Algorithm from scratch, without relying on existing libraries, in order to gain a deeper understanding of its internal mechanisms. Particular emphasis is placed on the formulation of an effective fitness function and the tuning of algorithm parameters, as these factors play a critical role in determining the convergence speed and overall performance of the algorithm.

2. Background on Genetic Algorithms

2.1 Steps to create a genetic algorithm

1. Initial population.
2. Fitness function
3. Selection
4. Crossover & Mutation.
5. Repeat steps 3 & 4.

2.2 Parameters

1. Population size (Variable)
2. Mutation rate (Variable)
3. Crossover rate (Constant)
4. Number of generations (Constant)

3. Problem Formulation

The objective of this project is to simulate a password-breaking attempt by using a Genetic Algorithm (GA) to "guess" a specific target passcode. The passcode is a randomly generated sequence of 32 binary bits. The GA must evolve a population of candidate sequences through successive generations until an exact match is found.

3.1 Solution Representation

The solution is represented using **Binary Encoding**. Since the target is a sequence of bits, each potential solution in our algorithm is represented as a fixed-length string of 0s and 1s.

3.2 Chromosome Representation

In this Genetic Algorithm, a **Chromosome** represents one complete "potential passcode" or a single potential solution within the population.

- **Length:** Each chromosome is exactly 32 bits long.
- **Structure:** $C = [a_1, a_2, a_3, a_4, \dots, a_{32}]$ where each a_i is either 0 or 1.

3.3 Gene Representation

A **Gene** is the smallest unit of the solution.

- In this project, a single gene represents a **single binary bit** at a specific position in the 32-bit sequence.
- The value of a gene can be either **0** or **1**.

3.4 Search Space

The search space for this problem consists of all possible combinations of a 32-bit string.

- The total number of possible solutions is **$2^{(32)}$** , which equals **4,294,967,296** unique combinations.
- The Genetic Algorithm goal is to navigate this large search space efficiently to find the one specific target passcode without checking every possible combination.

3.5 Fitness Function Definition

The fitness function is the most critical part of the implementation. It measures how "close" a specific chromosome is to the target passcode.

Formulation: We define fitness as the number of bits in a candidate chromosome that match the bits in the target passcode at the exact same positions.

Mathematical Representation:

$$F(C) = \sum_{i=1}^{32} score(i)$$

where,

$$score(i) = 1 \text{ if } chromosome(i) = target(i)$$

$$score(i) = 0 \text{ if } chromosome(i) \neq target(i)$$

The fitness function was designed based on **Individual Bit Proximity**. We utilized a **Linear Fitness Scoring** mechanism where each matching bit increments the fitness

value. This provides a smooth **Fitness Landscape**, allowing the GA to avoid local optima and converge effectively towards the global solution (32 bits).

4. Implementation Details

The project was developed using **Java** and the **JavaFX** library for the graphical user interface. The implementation follows an Object-Oriented Programming (OOP) approach.

4.1. Core Components

- **Chromosome Class:** This class represents a potential solution. It consists of an integer array (genes) of length 32. It also stores the **fitness value**, which determines the quality of the solution.
- **GeneticAlgorithm Class:** This is the engine of the project. It handles the evolution loop, including selection, crossover, and mutation.

4.2. Evolutionary Stages in Code

- **Initialization**

The algorithm starts by generating a random population of chromosomes. Each gene is randomly assigned a value of 0 or 1.

Implementation: `initializePopulation()` method creates an `ArrayList<Chromosome>` with the size specified by the user in the UI.

- **Fitness Function**

The fitness function is the most critical part. It measures the "signal strength" of each chromosome by counting how many bits match the target passcode.

Implementation: `calculateFitness(int[] target)` iterates through the genes and increments a counter for every match: $\text{Fitness} = \sum (\text{gene}[i] == \text{target}[i])$.

- **Selection (Tournament Selection)**

To select parents for the upcoming generation, we used tournament selection. A small group of chromosomes is picked at random, and the one with the highest fitness is selected. This provides a balance between selecting the best individuals and maintaining genetic diversity.

- **Crossover (Single-Point)**

To create an offspring, we combine the genetic material of two parents. We pick a random point in the bit-string; genes before this point come from Parent A, and genes after it come from Parent B.

- **Mutation**

This stage introduces random changes to the offspring to prevent the population from getting stuck in local optima.

Implementation: For each gene in the child, a random number is generated. If it is less than the `mutationRate`, the bit is flipped (0 to 1 or 1 to 0).

4.3. Multi-Threading & UI Synchronization

To ensure the GUI remains responsive during the intense calculations of the GA, the algorithm runs on a **background Thread**. We used `Platform.runLater()` to safely update the JavaFX `LineChart` and `TextArea` in real-time as new generations are produced.

4.4. Termination Conditions

The algorithm is designed to stop under two conditions:

1. **Success:** If a chromosome reaches a fitness of **32/32**.
2. **Safety Timeout:** If the algorithm reaches **200 generations** without finding the exact passcode (to handle cases of extreme mutation rates).

5. Parameter Tuning Analysis

This section presents a comprehensive evaluation of the Genetic Algorithm's performance using a number of controlled experiments. By systematically varying the **Mutation Rate (M)** and **Population Size (P)**, we aimed to identify the optimal configuration for achieving the maximum fitness of 32.

The **Fitness Convergence Curves** and the final **Execution Statistics** for each scenario are shown in the following screenshots, which depict the algorithm's real-time behavior.

5.1 Experiment 1:

- M = 0.1
- P = 100

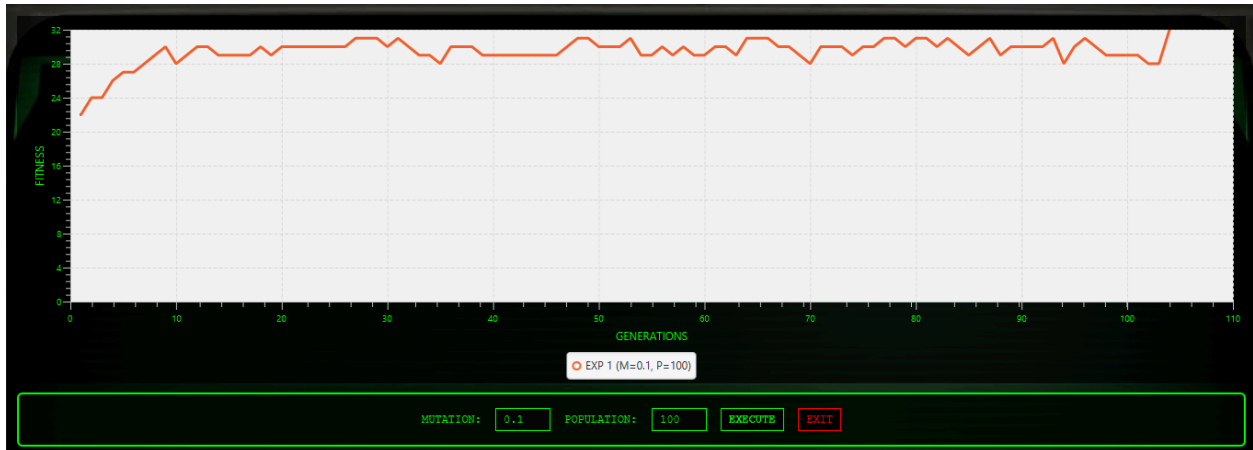


Figure 5.1.1 : Fitness Convergence for experiment 1

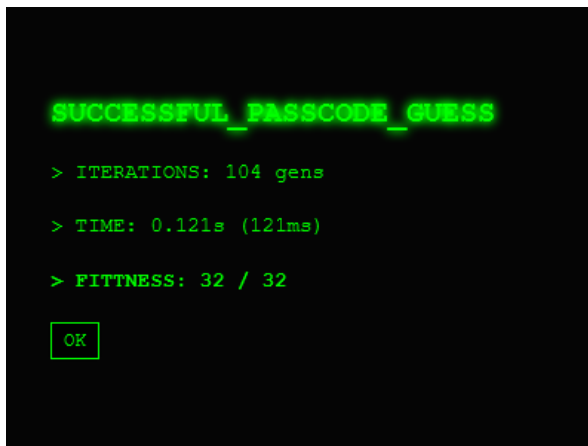


Figure 5.1.2 : Statistics for experiment 1

5.2 Experiment 2

- M= 0.01
- P = 100



Figure 5.2.1 : Fitness Convergence for experiment 2



Figure 5.2.2 : Statistics for experiment 2

5.3 Experiment 3

- $M = 0.0001$
- $P = 100$

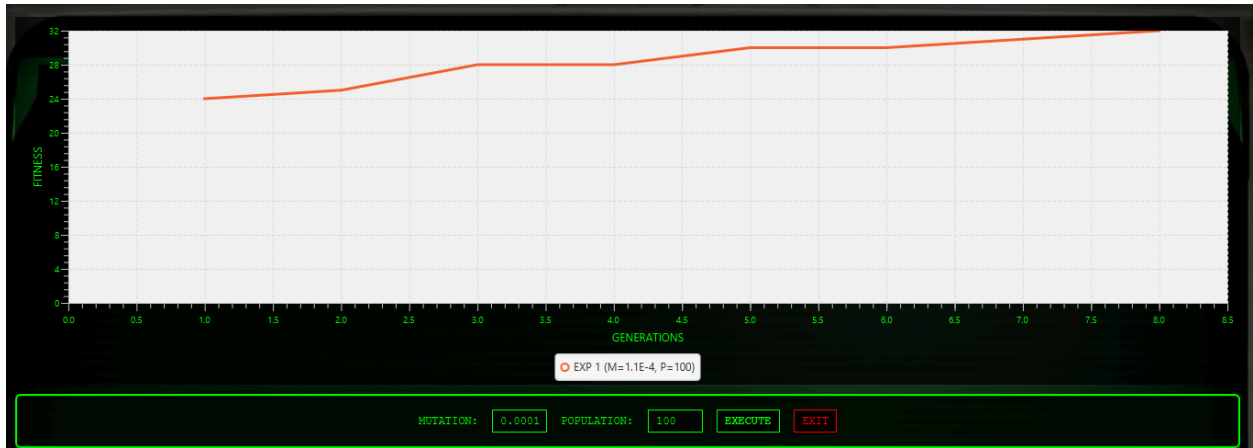


Figure 5.3.1 : Fitness Convergence for experiment 3

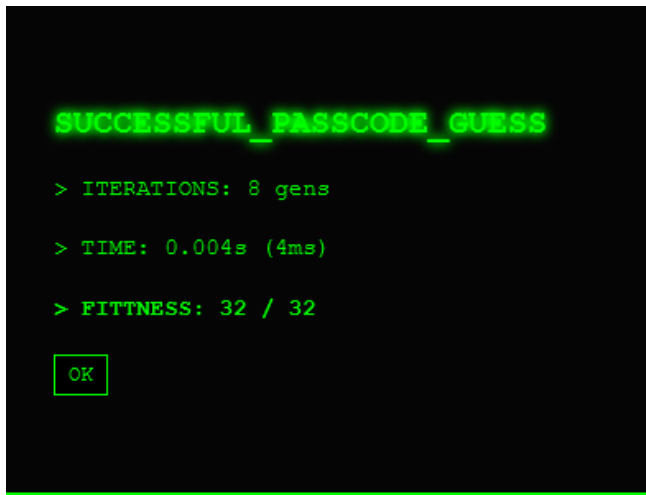


Figure 5.3.2 : Statistics for experiment 3

5.4 Experiment 4

- M= 0.01
- P = 300

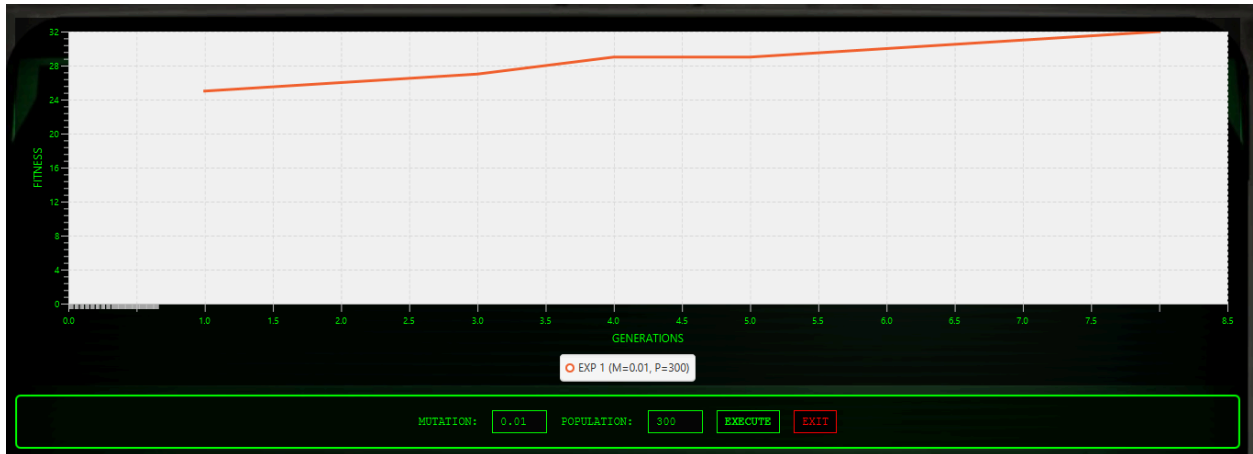


Figure 5.4.1 : Fitness Convergence for experiment 4

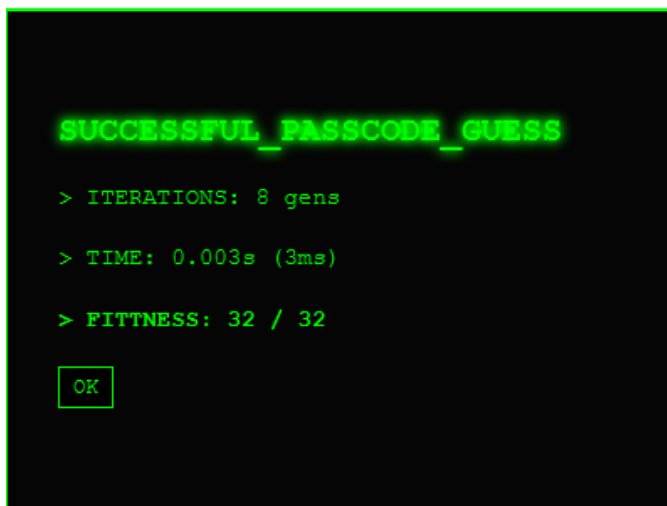


Figure 5.4.2 : Statistics for experiment 4

5.5 Experiment 5

- M = 0.001
- P = 600



Figure 5.5.1 : Fitness Convergence for experiment 5

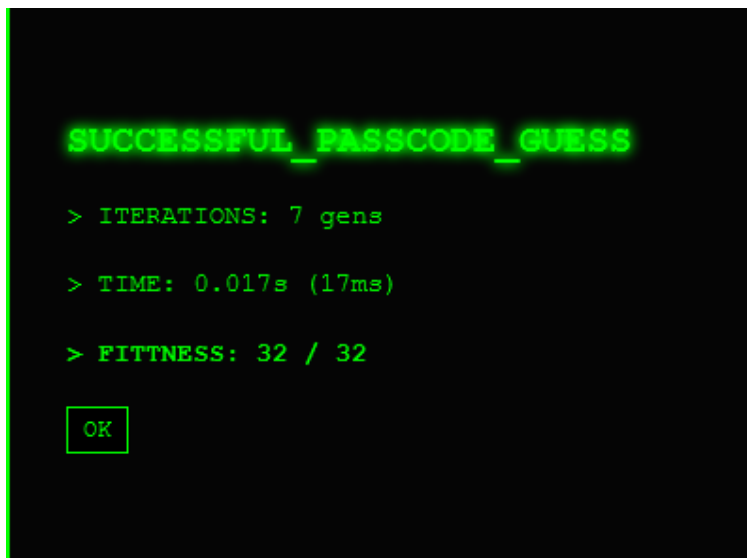


Figure 5.5.2 : Statistics for experiment 5

5.6 Experiment 6

- M= 1
- P = 100



Figure 5.5.3 : Fitness Convergence for experiment 6

6. Results and Discussion

In this section, we analyze the performance of the Genetic Algorithm (GA) based on the five experiments conducted. The main objective was to see how the **population size (P)** and **mutation rate (M)** affected the computational efficiency (execution time) and convergence speed (number of generations).

6.1 Analysis of Mutation Rate Impact (Exp 1, 2, 3, and 6)

With a constant population size ($P = 100$), the mutation rate proved to be the most critical factor for success:

- **Balanced Mutation (Exp 2 - $M=0.01$):** This experiment showed the most reliable convergence. The rate of mutation was both low enough to maintain the best genetic traits and high enough to introduce variety (Exploitation).
- **Low Mutation (Exp 3 - $M=0.0001$):** The algorithm struggled due to a lack of genetic diversity. Relying almost exclusively on crossover, the population risked stalling in local optima, leading to a slower climb toward the target.
- **High Mutation (Exp 1 - $M=0.1$):** Exploration increased significantly. While the algorithm searched the space quickly, the high frequency of random changes caused some instability in the fitness curve.
- **Extreme Mutation (Exp 6 - $M=1.0$):** This experiment is an example of a "**Total Random Search**". The algorithm was unable to maintain any progress because each gene was altered in each generation. As shown in Figure 5.6.1, the fitness fluctuated chaotically without a clear upward trend. As a result, the algorithm was unable to locate the target after reaching the **200-generation limit**.

6.2 Analysis of Population Size Impact (Exp 2, 4, and 5)

Increasing the population size generally improved the "intelligence" of the search:

- **Small Population (Exp 2 - P=100):** Reached the goal but required more generations to find the right combination of genes.
- **Large Populations (Exp 4 & 5 - P=300/600):** These experiments reached the target fitness of 32 in significantly fewer generations. A larger pool of individuals provides a better sampling of the search space at the very beginning (Generation 0).
- **The Trade-off:** While Exp 5 (P=600) converged in the fewest generations, it required more computational effort per generation. However, in this specific problem, the growing population helped to keep it well below the 200-generation mark.

6.3 Key Findings and Observations

1. **The Generation Limit:** The 200-generation limit acted as a necessary safety "timeout." Experiments that failed to converge (like Exp 6) demonstrated that without a proper mutation/population balance, the algorithm can run indefinitely without finding a solution.
2. **Exploration vs. Exploitation:** Success in GA depends on the balance between searching new areas (Exploration via Mutation) and refining existing good solutions (Exploitation via Crossover).
3. **Optimal Parameters:** Based on the data, the most effective configuration for this problem was a population size of **300-600** with a mutation rate of **0.01**.

6.4 Conclusion

The results confirm that the Genetic Algorithm is not a "magic" solver but a sensitive optimization tool. High mutation rates (Exp 6) destroy progress, while very low rates (Exp 3) slow it down. The population size acts as a buffer; larger populations provide faster convergence in terms of generations but increase the processing load.

7. Screenshots

7.1 Main Application Window (Welcome Window)

A welcome screen featuring the project title. Its to provide a clean transition into the simulation environment.

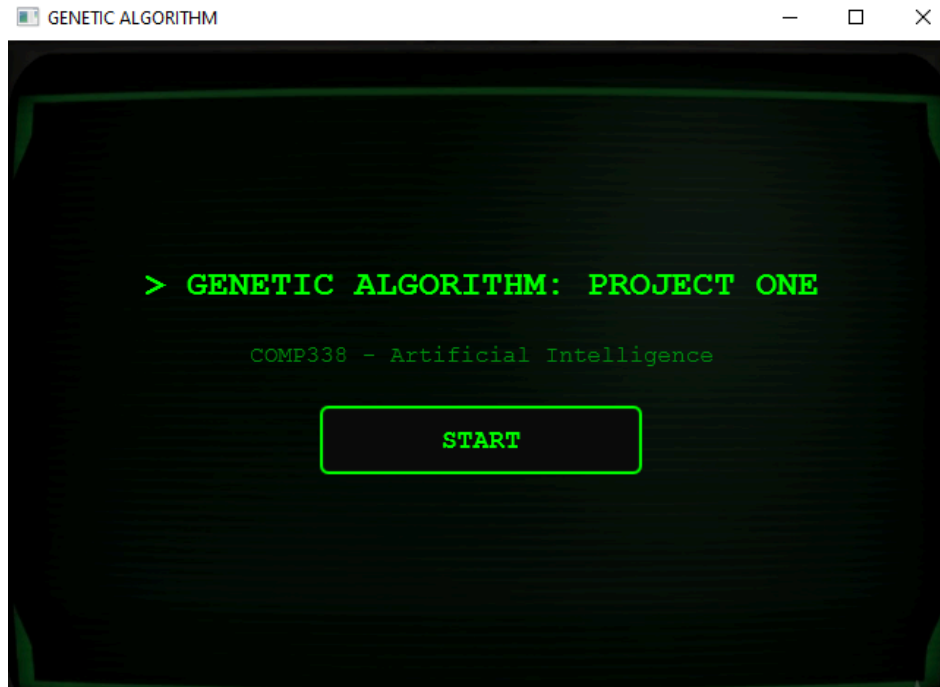


Figure 7.1 : Main Window

7.2 Simulation (Main App)

The core interface where the user inputs parameters (M and P). It displays the real-time fitness graph and a scrolling log of the evolutionary progress.

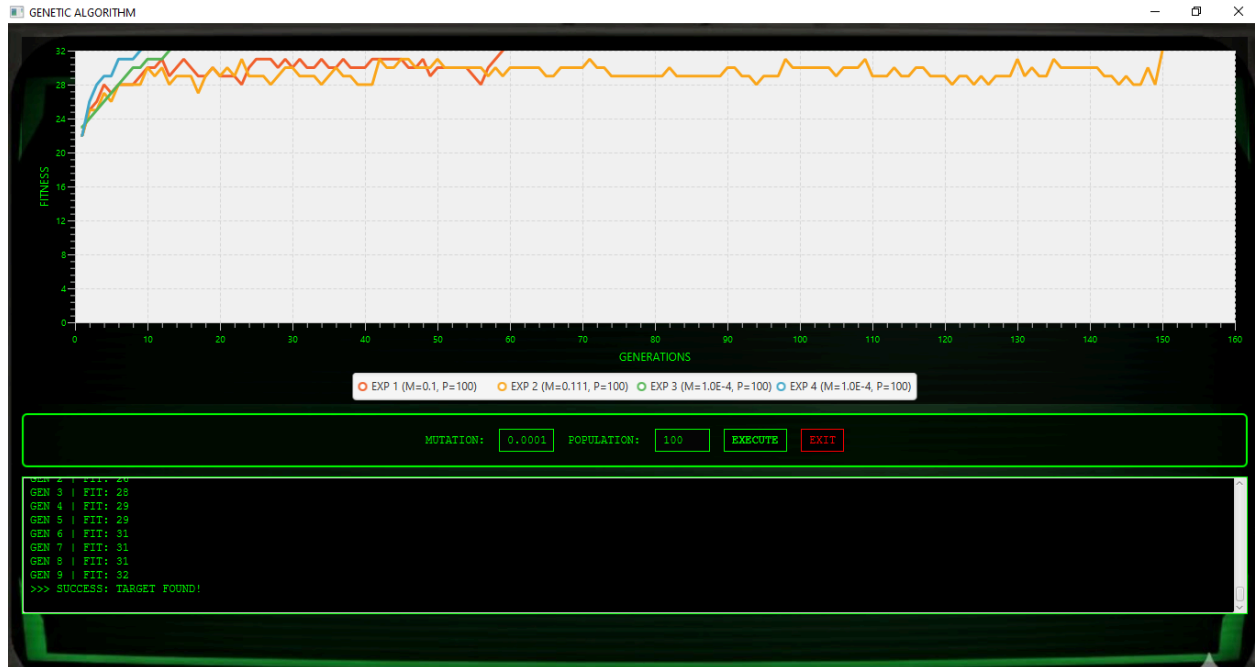


Figure 7.2 : Simulation Window

7.3 Report (Statistics Window)

A final results pop-up that summarizes the experiment's success, showing the exact time taken and the generation count where the target was found.

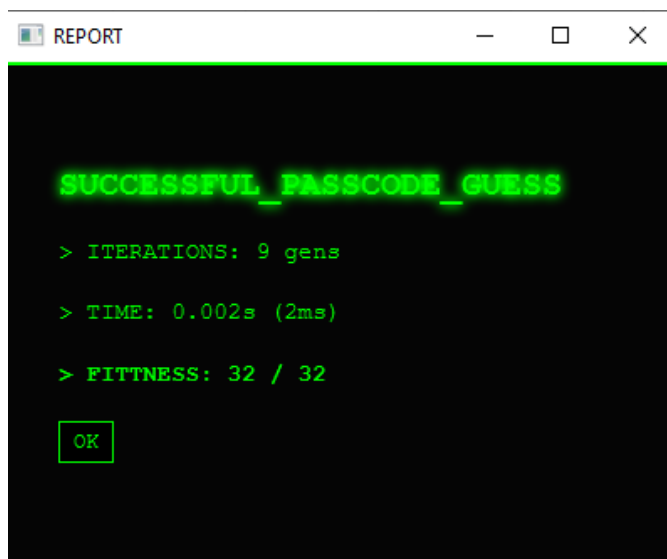


Figure 7.3 : Statistics Window

7.4 run() Method

The main engine of the algorithm. It manages the evolution loop, checks for the target solution, and ensures the process stops at the **200-generation limit**.

```
1 usage
public void run() {
    targetPasscode = generateRandomPasscode(length: 32);
    initializePopulation();

    boolean found = false;
    while (!found && generationCount < 200) { // So it doesn't get stuck STOP
        generationCount++;
        for (Chromosome c : population) {
            c.calculateFitness(targetPasscode);
            if (c.getFitness() == 32) found = true;
        }

        int bestFitness = getBestFitness();
        if (updateUI != null) updateUI.accept(generationCount, bestFitness);

        if (!found) evolve();
    }
}
```

Figure 7.4 : Running Method

7.5 tournamentSelection()

Selects the best parents for reproduction by picking a random group of individuals and choosing the one with the highest fitness score.

```
2 usages
private Chromosome tournamentSelection() {
    Random r = new Random();
    Chromosome best = null;
    for (int i = 0; i < 5; i++) {
        Chromosome ind = population.get(r.nextInt(population.size()));
        if (best == null || ind.getFitness() > best.getFitness()) best = ind;
    }
    return best;
}
```

Figure 7.5 : tournamentSelection Selection Method

7.6 crossover(Chromosome partner)

Simulates biological reproduction by picking a random midpoint and combining the first part of one parent with the second part of another to create a new child.

```
1 usage
public Chromosome crossover(Chromosome partner) {
    int[] childGenes = new int[32];
    Random random = new Random();

    // pick random point for crossover
    int midpoint = random.nextInt(bound: 32);

    // combine parents genes
    for (int i = 0; i < 32; i++) {
        if (i < midpoint) {
            childGenes[i] = this.genes[i]; // from current parent
        } else {
            childGenes[i] = partner.getGenes()[i]; // from the other parent
        }
    }

    return new Chromosome(childGenes);
}
```

Figure 7.6 : crossover method

7.7 mutate(double mutationRate)

Introduces genetic diversity by flipping bits (0 to 1 or vice versa) based on a small probability, preventing the algorithm from getting stuck.

```

// Mutation: flip bits with given probability
1 usage
public void mutate(double mutationRate) {
    Random random = new Random();

    for (int i = 0; i < genes.length; i++) {
        if (random.nextDouble() < mutationRate) {
            genes[i] = (genes[i] == 0) ? 1 : 0;
        }
    }
}

```

Figure 7.7 : Mutation Method

7.8 calculateFitness(int[] passcode)

Compares the chromosome's genes with the target passcode bit-by-bit. The fitness score is the total number of matching bits.

```

// Calculate fitness by comparing with passcode
1 usage
public void calculateFitness(int[] passcode) {
    int score = 0;

    for (int i = 0; i < genes.length; i++) {
        if (genes[i] == passcode[i]) {
            score++;
        }
    }

    this.fitness = score;
}

```

Figure 7.8 : Fitness Calculation Method

8. References

1.  Genetic algorithms explained in 6 minutes (...and 28 seconds)
2. [GeeksForGeejks - Genetic Algorithm](#)